

CRYSTAL DASH

Chapter 2: Collect the Crystals

Make objects talk to each other.
Turn a pickup into a score.



YOUR MISSION

Build a crystal that disappears when touched, send a signal to Main, and collect five crystals to reach a score of five.

START HERE

Continue your Chapter 1 project. Your robot should already move using WASD and arrow keys. Ask your mentor for crystal.svg before starting.

NAME _____ DATE _____

Godot 4.7 course • Chapter 2 student guide

Core route: pages 2-7 | Optional workshop: pages 8-9

1. Build your collectible

A crystal is something to collect. It should detect the robot without blocking its movement. That makes an Area2D the right starting point.

Create the Crystal scene

1. Stop the game with **F8**. Press **Ctrl+N**, choose **Other Node**, and create an Area2D.
2. Rename the root Crystal and save it as `crystal.tscn`.
3. Copy the supplied `crystal.svg` into your project folder, beside `project.godot`. Return to Godot and wait for it to appear in FileSystem.
4. With Crystal selected, drag `crystal.svg` into the 2D workspace. Rename the new Sprite2D node `CrystalSprite`.
5. Select `CrystalSprite`. In **Inspector > Transform**, set **Position** to (0, 0) and **Scale** to (0.40, 0.40).
6. Select Crystal. Add a `CollisionShape2D` child. Beside **Shape**, choose **New CircleShape2D**.
7. Click the **CircleShape2D** resource name to expand its settings. Set **Radius** to 22. Leave this node at Position (0, 0) and Scale (1, 1).

SCENE TREE

● Crystal	Area2D
● CrystalSprite	Sprite2D
● CollisionShape2D	CollisionShape2D

CHECKPOINT

The circle and crystal picture share the same centre. Both children sit directly under Crystal. Save with Ctrl+S.

THINK ABOUT IT

The picture shows the crystal. The circle detects a touch. What would happen if the circle were much bigger than the picture?

Asset tip: this PDF also contains `crystal.svg` as an attachment. In a PDF reader with an Attachments panel, save it to your project folder. Your mentor can supply it separately if your viewer does not show attachments.

2. Touch it. Make it vanish.

Godot already gives Area2D a signal called `body_entered`. It fires when a physics body enters the detection area. We will connect it to a function on Crystal.

1. Select Crystal and attach an empty **GDScript** at `res://crystal.gd`. If a template supplies extra code, replace it with `extends Area2D`.
2. Select Crystal again. Open **Node > Signals** on the right (some layouts show a separate **Signals** tab).
3. Double-click `body_entered(body: Node2D)`. Choose **Crystal** as the receiver, keep `_on_body_entered` as the method name, and click **Connect**.
4. Godot adds a function. Replace its pass line. Your whole script should now match this:

```
extends Area2D

func _on_body_entered(body: Node2D) -> void:
    if body.name == "Player":
        queue_free()
```

`body` tells us what entered. The `if` checks its name. `queue_free()` schedules this Crystal and its children for removal.

5. Save the scene. Open `main.tscn` and drag `crystal.tscn` into it. Set the Crystal instance Position to **x 700, y 270**.
6. Keep the robot instance named exactly `Player`. Save and press **F6** while Main is open. Drive into the crystal.

CHECKPOINT

The crystal disappears when touched. Stop with F8 and run again: it returns because you are starting a fresh copy of the saved scene.

SEEING THE COLLISION CIRCLES?

In the editor they help you line things up. During play, toggle **Debug > Visible Collision Shapes**. Turning this display off does not turn collisions off.

THINK ABOUT IT

Why do we check which body entered? What other moving objects might we add to the game later?

3. Announce the collection

The crystal can disappear, but Main needs to know a pickup happened. Let us create a signal with a name that describes that event: `collected`.

1. Open `crystal.gd`. Add signal `collected` below the `extends` line.
2. Add `collected.emit()` immediately before `queue_free()`, at the same indentation level.

The complete script is now:

```
extends Area2D

signal collected

func _on_body_entered(body: Node2D) -> void:
    if body.name == "Player":
        collected.emit()
        queue_free()
```

Line	What it does
<code>signal collected</code>	Declares an event that this Crystal can announce.
<code>collected.emit()</code>	Sends that event to any connected listeners.
<code>queue_free()</code>	Removes this Crystal after it has announced the pickup.

A SCRATCH CONNECTION

This is similar to “broadcast `collected`” followed by “delete this clone”. In Godot, a listener must connect to the signal on the particular node sending it.

CHECKPOINT

Save and play again. The crystal still disappears. Nothing else changes yet: Main has not been connected as a listener.

THINK ABOUT IT

A signal announces what happened. The listener decides what to do. What else could listen for `collected` besides a score counter?

My idea: _____

TWO SIGNALS, TWO JOBS

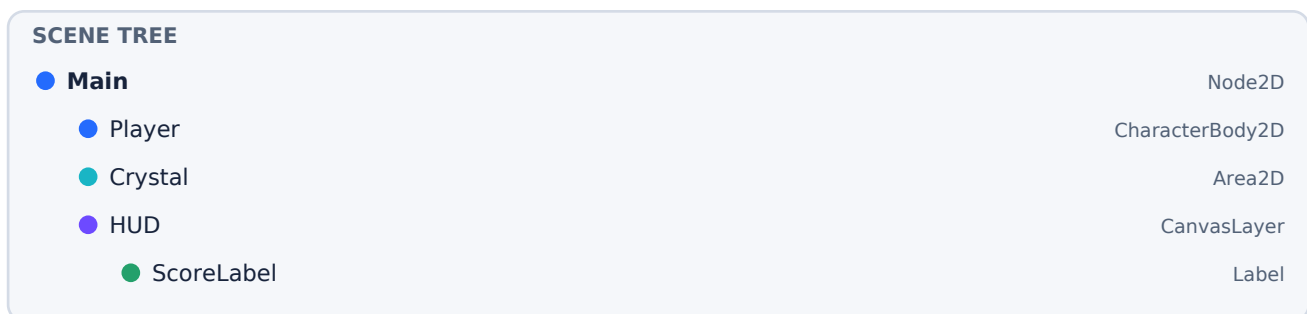
`body_entered` is built into `Area2D` and reports a body entering.
`collected` is our custom signal and reports a successful pickup.

4. Give the score a home

The score belongs to Main, which stays alive after a crystal disappears. We will show it in a Label on a HUD: a heads-up display.

Build the display in main.tscn

1. Stop the game and open `main.tscn`. Select Main, add a **CanvasLayer** child, and rename it HUD.
2. Select HUD, add a **Label** child, and rename it `ScoreLabel`. Use these names exactly.
3. Set the Label's **Text** to `Crystals: 0`. Under **Layout > Transform**, set Position to **x 24, y 20**.
4. Under **Theme Overrides > Font Sizes**, enable **Font Size** and enter **28**.



CanvasLayer draws the HUD above the game world with its default settings. It is useful for a score that stays on screen if we later add a moving camera.

Give Main a script

5. Select Main. Attach an empty GDScript named `res://main.gd`. Enter:

```
extends Node2D

var score: int = 0

@onready var score_label: Label = $HUD/ScoreLabel
```

`score` stores a whole number. `@onready` waits for the scene's nodes to be ready. `$HUD/ScoreLabel` finds the Label by following its path from Main.

CHECKPOINT

Save and run Main with F6. You should see "Crystals: 0". It will not increase until the next page.

THINK ABOUT IT

What might go wrong if we stored the running total inside the crystal that is about to disappear?

5. Listen and add a point

We are connecting the Crystal **instance inside Main** to Main's script. This is a different connection from the `body_entered` signal inside `crystal.tscn`.

1. Stop the game. In `main.tscn`, select the existing **Crystal** instance.
2. Open **Node > Signals** (or the Signals tab). Double-click `collected()`.
3. Choose **Main** as the receiving node. Use `_on_crystal_collected` as the method name and click **Connect**.
4. Replace the generated pass with the two lines below. Keep just one function of this name. Your complete `main.gd` is:

```
extends Node2D

var score: int = 0

@onready var score_label: Label = $HUD/ScoreLabel

func _on_crystal_collected() -> void:
    score += 1
    score_label.text = "Crystals: " + str(score)
```

Code	Scratch-style meaning
<code>score += 1</code>	Change score by 1.
<code>str(score)</code>	Turn the number into text.
<code>"Crystals: " + str(score)</code>	Join "Crystals: " and the score.
<code>score_label.text = ...</code>	Replace the words shown in the Label.

CHECKPOINT

Save Main. Press F6. Touch the crystal: it disappears and the label changes from "Crystals: 0" to "Crystals: 1".

THINK ABOUT IT

What do you predict would happen if you changed `score += 1` to `score += 2`? Try it, then restore 1 for our five-crystal test.

THE WHOLE JOURNEY

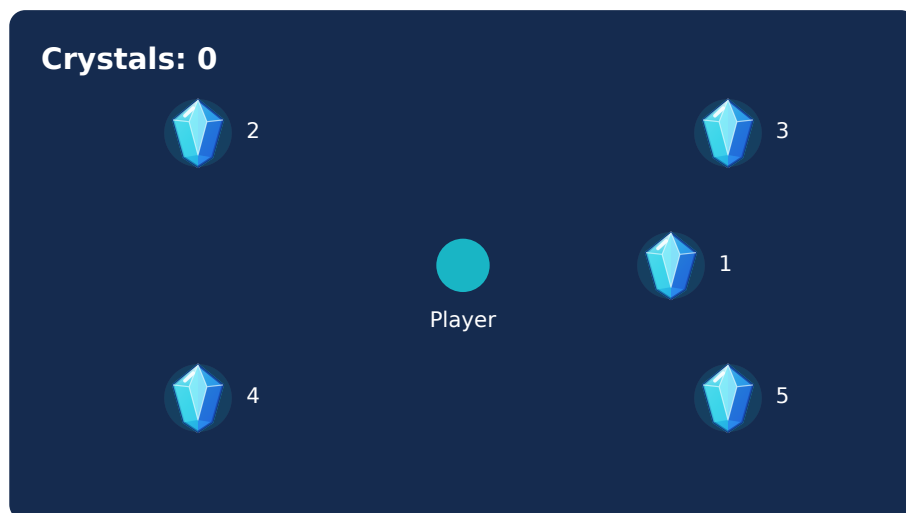
Player touches Crystal. Crystal emits `collected`. Main receives it, adds one to score and updates `ScoreLabel`. Crystal then removes itself.

6. One crystal becomes five

An **instance** is a copy of a saved scene placed in your game. Each crystal can be collected separately, while all five use the same scene and script.

1. Stop the game and open Main. Select the **already-connected Crystal** in the Scene tree. Press **Ctrl+D** four times to make five crystals in total.
2. Select each instance and set its **Transform > Position** using the table. Move the root of the instance so its picture and detection shape move together.

Instance	X	Y
Crystal	700	270
Crystal2	200	130
Crystal3	760	130
Crystal4	200	410
Crystal5	760	410



Layout sketch: the numbers identify the five crystals. They are not extra labels to add to the game.

3. Save and run Main. Collect all five in any order. Every pickup should add exactly one point.

CHECKPOINT

All five crystals disappear, one at a time. Your score reaches “Crystals: 5”. Chapter 2 core mission complete!

IF A COPY DOES NOT SCORE

Duplicating the connected instance should copy its signal connection. Select the affected copy in Main and inspect collected in Signals. If no connection is listed, connect it to Main using the existing method `_on_crystal_collected`. Do not create a separate scoring function for each copy.

7. Red is helpful!

Godot checks GDScript while you type. An unfinished line can turn red temporarily. **Finish the line first.** If the error remains, read the first error message and fix one thing at a time.

THE THREE-STEP DEBUGGING HABIT

1. Read the first error, even if several appear.
2. Check spelling, quotes, brackets, the colon and indentation.
3. Correct it and watch whether the later errors disappear.

Practice being a Code Detective

Before running, temporarily remove the colon at the end of `if body.name == "Player":` in `crystal.gd`. Look at Godot's feedback. Put the colon back and check that the error clears. Save the repaired script.

What you see	What to check
Crystal will not disappear	Is <code>body_entered</code> connected inside <code>crystal.tscn</code> ? Is the robot instance named <code>Player</code> ? Does the circle overlap the artwork? Keep the default collision layer/mask settings for this chapter.
Crystal disappears; score stays at 0	Check the collected connection from that instance to <code>Main</code> . Make sure <code>collected.emit()</code> is present in <code>crystal.gd</code> .
Error mentions <code>ScoreLabel</code> or a null value	The scene tree must contain <code>Main / HUD / ScoreLabel</code> . Node names and capital letters must match <code>\$HUD/ScoreLabel</code> .
Score increases by two	Restore <code>score += 1</code> . Check for two crystals in the same place and for extra scoring connections or code.
No robot or score when you run	Open <code>main.tscn</code> before pressing <code>F6</code> . <code>F6</code> runs the currently selected scene.
Pickup triggers far from its picture	In <code>crystal.tscn</code> , set <code>CrystalSprite</code> and <code>CollisionShape2D</code> <code>Position</code> to <code>(0, 0)</code> . If the robot has the same issue, centre its sprite on its collision shape too.

THINK ABOUT IT

A script can have no typing errors and still behave incorrectly. Can you think of an example from this chapter?

My example: _____

Not every error appears while typing. Missing node paths and other runtime problems may appear only when you play. Stop, read the message, and investigate the named node or line.

8. Level Up workshop

These challenges are optional. Complete the five-crystal test first, then choose one experiment. Predict, change, play, and explain what happened.

LEVEL UP: Design a collection route

Reposition the five crystals within the 960 x 540 arena. Keep them away from the HUD and starting robot. Ask a friend to collect them. Which route do they choose?

LEVEL UP: Make the finish feel like a finish

With exactly five crystals worth one point each, replace only your `_on_crystal_collected` function in `main.gd` with this version:

```
func _on_crystal_collected() -> void:
    score += 1
    score_label.text = "Crystals: " + str(score)

    if score == 5:
        score_label.text = "All 5 crystals collected!"
```

On the fifth pickup, the label celebrates your finish. This adds a completion message; the robot can still move. If you change the number of crystals or their point values, this fixed target must change too.

THINK ABOUT IT

One equals sign assigns a value. Two equals signs compare values. What is the difference between `score = 5` and `score == 5`?

LEVEL UP: Record your milestone

Record 10-20 seconds showing a pickup and the score changing. In Windows 11, Snipping Tool can record just the game area. Your mentor can help save it as `02_crystal_collection.mp4`.

CHECKPOINT

- [] I can explain what `body_entered` detects.
- [] I can explain what `collected` announces.
- [] Every crystal updates the shared score.
- [] I can find and fix a simple script error.
- [] I have saved my scenes and scripts.

NEXT: Add danger and a reason to hurry

Chapter 3 will build on our collection game with enemies and a countdown. Keep your project folder ready for the next session.

Reference links for mentors

[Godot: Using signals](#) • [Area2D](#) • [CanvasLayer](#)

Prepared 5 September 2026. Core steps follow our Godot 4.7.1 walkthrough; optional challenges extend that work.